Dynamic Programming

Reading:

CLRS: chapter 16 (1st ed.)

chapter 15 (2nd and 3rd ed.)

Review – Lecture 11

Max flow:

Time complexity of FF:

- A single iteration takes $O(|E|)$.
- If all the capacities are integers, then FF may need f^* iterations, where f^* is the value of the max flow.
- Edmonds-Karp (EK) algorithm: FF method, select in every iteration a shortest augmenting path (BFS from s till t is labeled).
- This guarantees at most $O(|E||V|)$ iterations.

So the time complexity of EK is $O(\min(|E|^2|V|), (f^*|E|))$

Review – Lecture 11

The last residual network can be used to solve infrastructure-related problems

Examples: Upward and Downward critical edges

Review – Lecture 11

Using max-flow to solve other problems

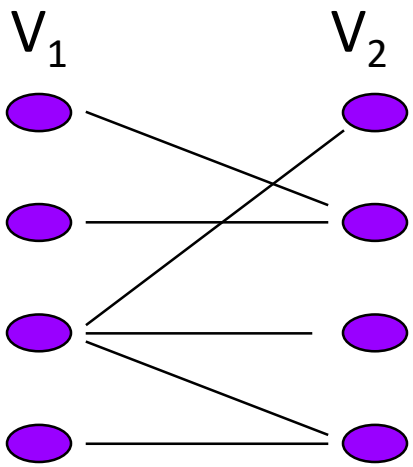
Many resource-allocation problems can be reduced to a max-flow problem.
Limited edge capacity correspond to limited availability of a resource, or limited demand by a client.

Example: Maximum matching in a bipartite.

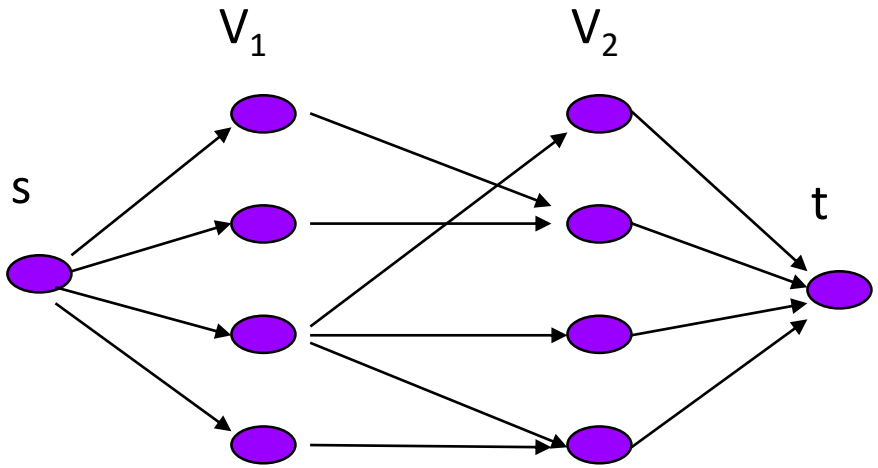
Review – Lecture 11

Example: Maximum matching in a bipartite.

$G = (V_1 \cup V_2, E)$



$N = (V', E')$



$V' = V_1 \cup V_2 \cup \{s,t\}$

$E' = E \cup \{(s \rightarrow u) \mid \forall u \in V_1\} \cup \{(v \rightarrow t) \mid \forall v \in V_2\}$

For all $e \in E'$, $c(e) = 1$.

G includes a matching of size k $\Leftrightarrow$ N has flow with value k.

Dynamic Programming

- A strategy for designing algorithms.
- A technique, not an algorithm.
- The word “programming” is historical and predates computer programming.
- Use when problem breaks down into recurring small sub-problems.

Recursive Solutions

- Divide a problem into smaller subproblems
- Recursively solve subproblems
- Combine solutions of subproblems to get solution to original problem
- Occasionally, straightforward recursive solution takes too much time, because the same subproblems are solved over and over again

Recursive Solutions

demo

- Example: Fibonacci Numbers

F(0) = 1 ; F(1) = 1

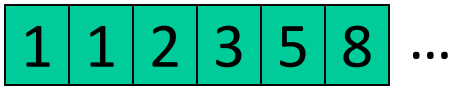
F(n) = F(n – 1) + F(n – 2)

Exponential running time

```
fib(n)
  if (n < 2)  return 1
  return fib(n-1) + fib(n-2)
```

Recursive Solutions

- How many different subproblems are there, for finding fib(n) ? Only n-1
- The recursion takes so much time because we are recalculating solutions subproblems over and over and over again.
- What if we stored solutions to subproblems in a table, and only recalculated if the values were not in the table?




```
Fibonacci(n)
T[0] = 1; T[1] = 1
for i = 2 to n do T[i] = T[i-1] + T[i-2]
return T[n]
```

Dynamic Programming

* Simple, recursive solution to a problem
* Straightforward implementation of recursion leads to exponential behavior, because of repeated subproblems
* Create a table of solutions to subproblems
* Fill in the table, in an order that guarantees that each time you need to fill in a square, the values of the required subproblems have already been filled in.

Example 1: Max-sum subsequence

- Given a sequence of numbers $a_1,a_2,\ldots,a_n$.
- The goal is to find a consequent sub-sequence whose sum is maximal.
- Example:

	1	2	3	4	5	6	7	8
a[i]	-2	6	-3	4	5	-18	3	4

- The answer is 12 - achieved by $a_2,\ldots,a_5$
- Can be solved in time $O(n^2)$ – by calculating $S_{ij} = a_i+\ldots+a_j$ for every pair $i \leq j$.

Max-sum subsequence

Linear time solution based on dynamic programming**:**

Define C[i] = maximal sum of a sub-sequence that ends in a_i and includes it.

Base: $C[1]=a_1$

Our goal: $\max_i C[i]$

Recursive formula: $C[i]= \max\{a_i, C[i-1]+a_i\}$

Time complexity: O(n)

	1	2	3	4	5	6	7	8
a[i]	-2	6	-3	4	5	-18	3	4
C[i]	-2	6	3	7	12	-6	3	7

Max-sum subsequence

In order to retrieve the indices:

Add another field specifying the leftmost index in the subsequence.

Base: left[1]=1

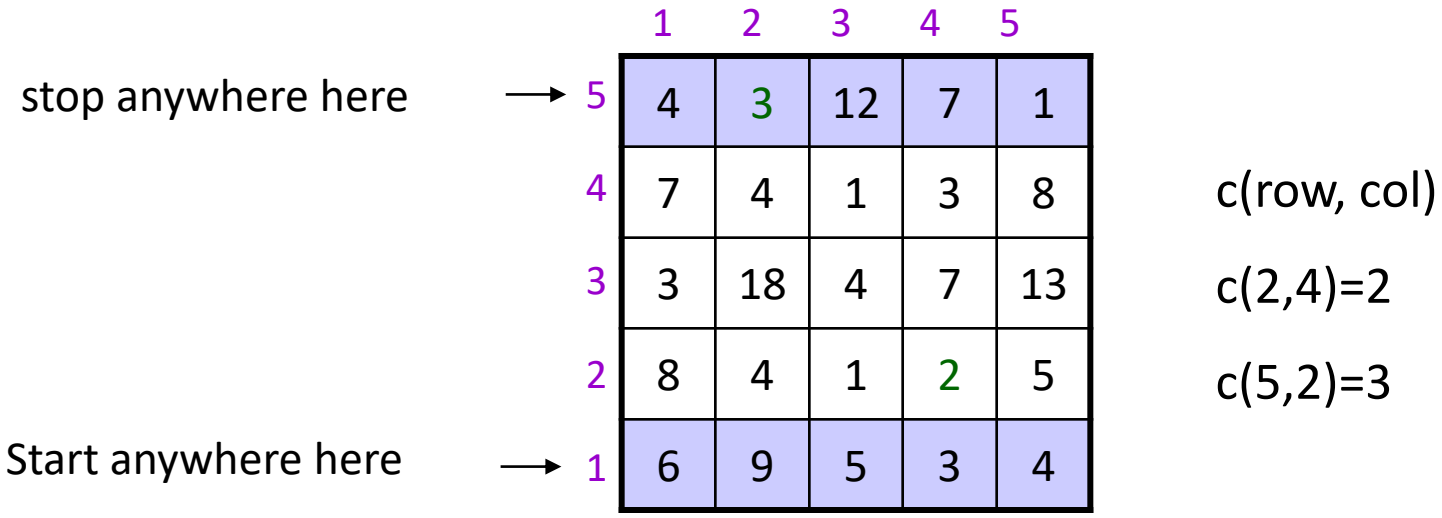
For i>1: if c[i]=a_i then left[i]=i, else left[i]=left[i-1]

	1	2	3	4	5	6	7	8
a[i]	-2	6	-3	4	5	-18	3	4
C[i]	-2	6	3	7	12	-6	3	7
Left[i]	1	2	2	2	2	2	7	7

Let i be the index such that c[i] is maximal, then the sequence starts in index left[i] and ends in index i.

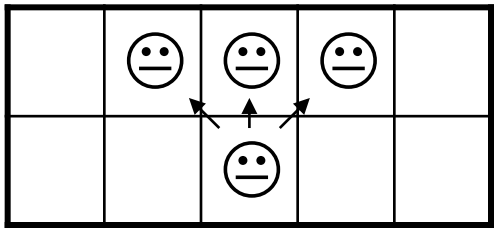
Example 2: Most Profitable Tour

- Assume that you need to travel from the bottom row of a chessboard to the top row. You can select your initial and final locations (anywhere on the bottom and top rows)
- On each square (i,j) there are c(i,j) coins.



Example: Most Profitable Tour

- Whenever you visit a square you can pick up the money on it.
- The amounts c(i,j) are known in advance.
- From each square you can advance to the next row in all three directions (diagonally left, diagonally right, or straight forward)
- You want to maximize your profit.



A possible tour.
Profit = 40

4	3	12	7	1
7	4	1	3	8
3	18	4	7	13
8	4	1	2	5
6	9	5	3	4

Most Profitable Tour

- Define $q(i,j)$ as the maximal possible profit to reach square (i,j).
- For any column j, $q(1,j) = c(1,j)$.
- For any column j and i>1,
  $q(i,j) = c(i,j)+\max\{q(i-1,j-1), q(i-1,j), q(i-1,j+1)\}$
- Make sure you don't leave the board:

  if $j<1$ or $j>n$ then $q(i,j) = -\infty$.
- The goal: find $\max_j q(n,j)$
- If the calculation of q is implemented by a recursive program: Exponential time.

Most Profitable Tour: DP solution

Filling the table:

- Init:

-∞						-∞
-∞						-∞
-∞						-∞
-∞						-∞
-∞	6	9	5	3	4	-∞

Input:

4	3	12	7	1
7	4	1	3	8
3	18	4	7	13
8	4	1	2	5
6	9	5	3	4

- Then fill row after row (bottom to top)

46	45	51	43	31
42	39	36	25	30
20	35	17	17	22
17	13	10	7	9
6	9	5	3	4

Dynamic Programming works!

```
function maxProfit( )  //fill the whole table!
for j = 1 to n
    q[1, j] = c(1, j)
for i = 1 to n
    q[i, 0] = -∞
    q[i, n + 1] = -∞
for i = 2 to n
    for j = 1 to n
        m = max(q[i-1, j-1], q[i-1, j], q[i-1, j+1])
        q[i, j] = m + c(i, j)
```

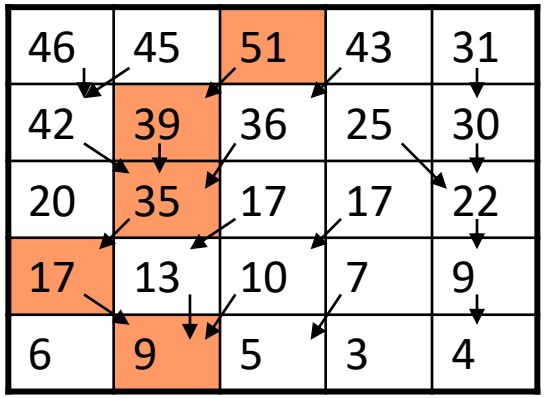
```
main()
    maxProfit()
    return max_j q[n,j]
```

Time complexity: $O(n^2)$

Most Profitable Tour: DP solution

Finding the actual path:

- Whenever m = max(q[i-1, j-1], q[i-1, j], q[i-1, j+1]) is calculated, remember the origin of the max value.



46	45	51	43	31
42	39	36	25	30
20	35	17	17	22
17	13	10	7	9
6	9	5	3	4

4	3	12	7	1
7	4	1	3	8
3	18	4	7	13
8	4	1	2	5
6	9	5	3	4

- In the code: **if** m = q[i-1, j-1] then d[i, j] = -1; **else if** m = q[i-1, j] then d[i, j] = 0; **else** d[i, j] = 1

Example 3: Longest Common Subsequence

- *Longest common subsequence* (*LCS*) problem:
  - Given two sequences x[1..m] and y[1..n], find the longest subsequence which occurs in both (not necessarily consequent).
  - Example: x = {A B C B D A B }, y = {B D C A B A}
  - {B C} and {A A} are both subsequences of both
    - What is the LCS? BCAB, BCBA

    (two solutions, both of length 4)

LCS Problem

- Brute-force algorithm: For every subsequence of x, check if it is a subsequence of y
    - How many subsequences of x are there?
    - What will be the running time?
- Brute-force algorithm: 2^m subsequences of x each takes O(n) to search in y: $O(n\,2^m)$
- We can do better: for now, let's only worry about the problem of finding the *length* of the LCS
  - When finished we will see how to backtrack from this solution back to the actual LCS.
- Notice LCS problem has optimal substructure
  - Subproblems: LCS of pairs of *prefixes* of x and y

Finding LCS Length

- Define c[i,j] to be the length of the LCS of X_i =x[1..i] and Y_j=y[1..j]
- What is the length of LCS of x and y?

  C[m,n]

- The recursive formula:

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \text{otherwise} \end{cases}$$

LCS recursive solution

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \text{otherwise} \end{cases}$$

When calculating c[i,j], there are two cases to consider:

- **First case:** x[i]=y[j]: one more symbol in strings X and Y matches, so the length of LCS X_i and Y_j equals to the length of LCS of smaller strings X_{i-1} and Y_{i-1} , plus 1.

LCS recursive solution

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \text{otherwise} \end{cases}$$

- **Second case:** $x[i] \neq y[j]$
- As symbols don't match, our solution is not improved, and the length of $LCS(X_i, Y_j)$ is the maximum of $LCS(X_i, Y_{j-1})$ and $LCS(X_{i-1}, Y_j)$

LCS recursive solution

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \text{otherwise} \end{cases}$$

Why not just take the length of LCS(X_{i-1}, Y_{j-1}) ?

Answer: Let x=abc y=db

c[3,2]=max(c(3,1), c(2,2))=max(0,1)=1

c[3,2]≠ c(2,1)=0

LCS Algorithm

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \text{otherwise} \end{cases}$$

- Start with i=j=0 (empty substrings of X and Y)
- Since X_0 and Y_0 are empty strings, their LCS is always empty (i.e. c[0,0] = 0)
- LCS of empty string and any other string is empty, so for every i and j: c[0, j] = c[i,0] = 0
- We can now proceed filling the table.

LCS Length Algorithm

LCS-Length(X, Y)

```
1.  m = length(X)                          // # of symbols in X
2.  n = length(Y)                          // # of symbols in Y
3.  for i = 1 to m:      c[i, 0] = 0       // special case: Y0
4.  for j = 1 to n:      c[0, j] = 0       // special case: X0
5.  for i = 1 to m:                        // for all Xi
6.      for j = 1 to n:                    // for all Yj
7.          if ( Xi == Yj ):
8.              c[i,j] = c[i-1, j-1] + 1
9.          else:
10.             c[i, j] = max ( c[i-1, j], c[i, j-1] )
11. return c[m, n]
```

LCS Example

We'll see how LCS algorithm works on the following example:

- X = ABCB
- Y = BDCAB

What is the Longest Common Subsequence of X and Y?

LCS(X, Y) = BCB

X = A **B** **C** **B**

Y = **B** D **C** A **B**

LCS Example (0)

ABCB

BDCAB

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi						
1	**A**						
2	**B**						
3	**C**						
4	**B**						

X = ABCB; m = |X| = 4

Y = BDCAB; n = |Y| = 5

Allocate array c[5,4]

LCS Example (1)

ABCB

BDCAB

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**					
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

for i = 1 to m c[i,0] = 0

for j = 1 to n c[0,j] = 0

LCS Example (2)

ABCB

BDCAB

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0				
2	B		0					
3	C		0					
4	B		0					

i=1

j=1

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (3)

ABCB

BDCAB

i		j	0	1	2	3	4	5
			Yj	**B**	**D**	**C**	**A**	**B**
0	Xi		**0**	**0**	**0**	**0**	**0**	**0**
1	**A**		**0**	**0**	**0**	**0**		
2	**B**		**0**					
3	**C**		**0**					
4	**B**		**0**					

i=1

j=2,3

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (4)

ABCB

BDCAB

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	
2	B	0					
3	C	0					
4	B	0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

i=1

j=4

LCS Example (5)

ABCB

BDCAB

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0					
3	C		0					
4	B		0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

i=1

j=5

LCS Example (6)

ABCB

BDCAB

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1				
3	C		0					
4	B		0					

i=2

j=1

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (7)

ABCB

BDCAB

i		j	0	1	2	3	4	5
		Yj		B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	
3	C		0					
4	B		0					

i=2

j=2,3,4

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (8)

ABCB

BDCAB

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**					
4	**B**	**0**					

i=2

j=5

if (X_i == Y_j)

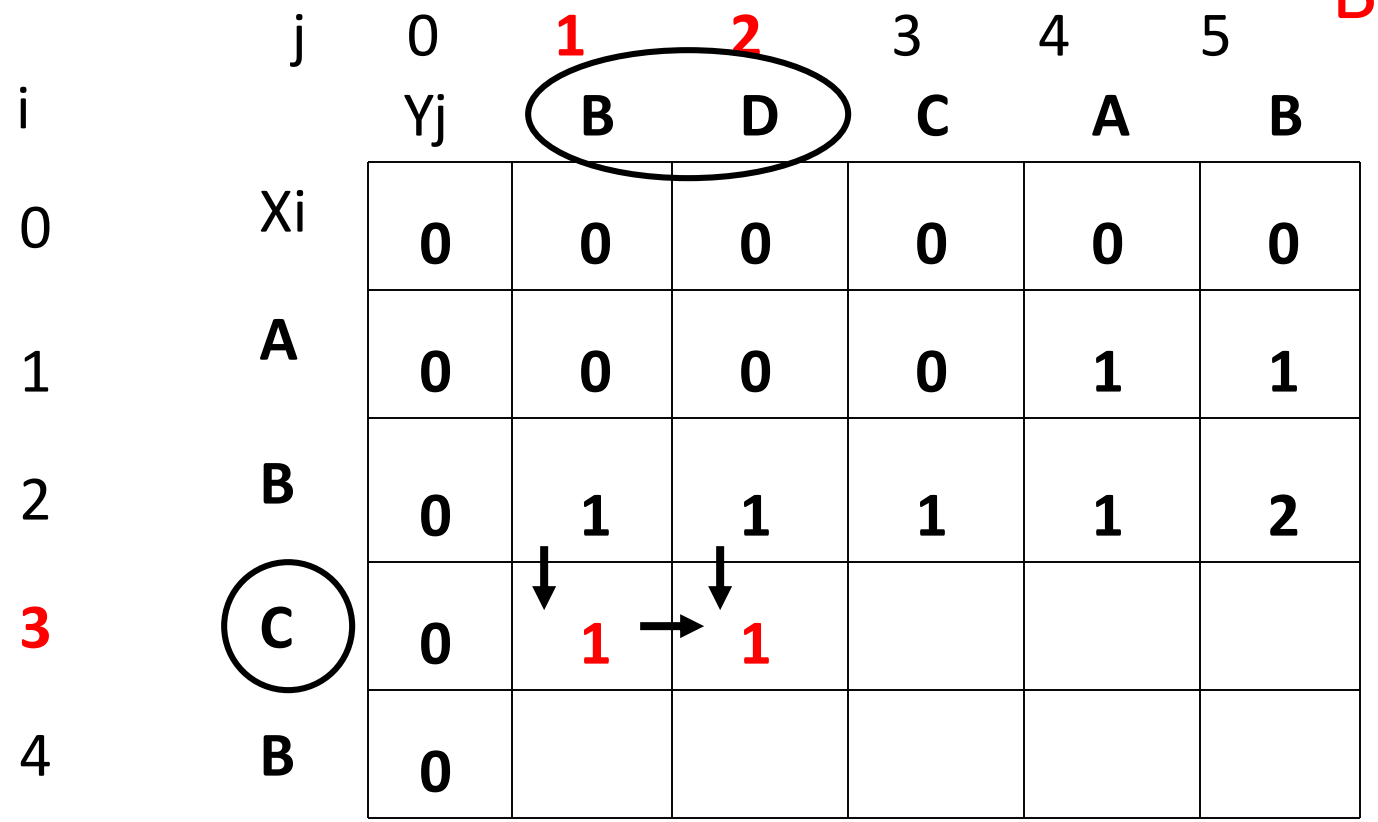
c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (9)

ABCB

BDCAB



i=3

j=1,2

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (10)

ABCB

BDCAB

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2		
4	B		0					

i=3

j=3

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (11)

ABCB

BDCAB

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0					

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

i=3

j=4,5

LCS Example (12)

ABCB

BDCAB

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1				

i=4

j=1

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (13)

ABCB

BDCAB

i		j	0	1	2	3	4	5
		Yj		B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1	1	2	2	

i=4

j=2,3,4

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Example (14)

ABCB

BDCAB

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	2
3	C	0	1	1	2	2	2
4	B	0	1	1	2	2	3

i=4

j=5

if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS Algorithm Running Time

- LCS algorithm calculates the values of each entry of the array c[m,n]
- So what is the running time?

O(mn)

Each c[i,j] is calculated in constant time, and there are (m+1)(n+1) elements in the array. m+n+1 base cases. mn other cases.

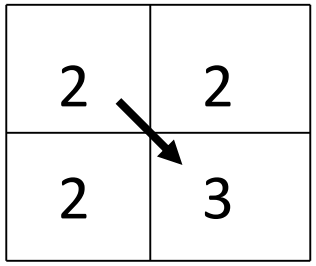
How to find actual LCS

- So far, we have just found the *length* of LCS, but not LCS itself.
- We want to modify this algorithm to make it output the Longest Common Subsequence of X and Y

Each c[i,j] depends on c[i-1, j] and c[i, j-1]

or c[i-1, j-1]

For each c[i, j] we can say how it was acquired:



For example, here

c[i, j] = c[i-1, j-1]+1 = 2+1 = 3

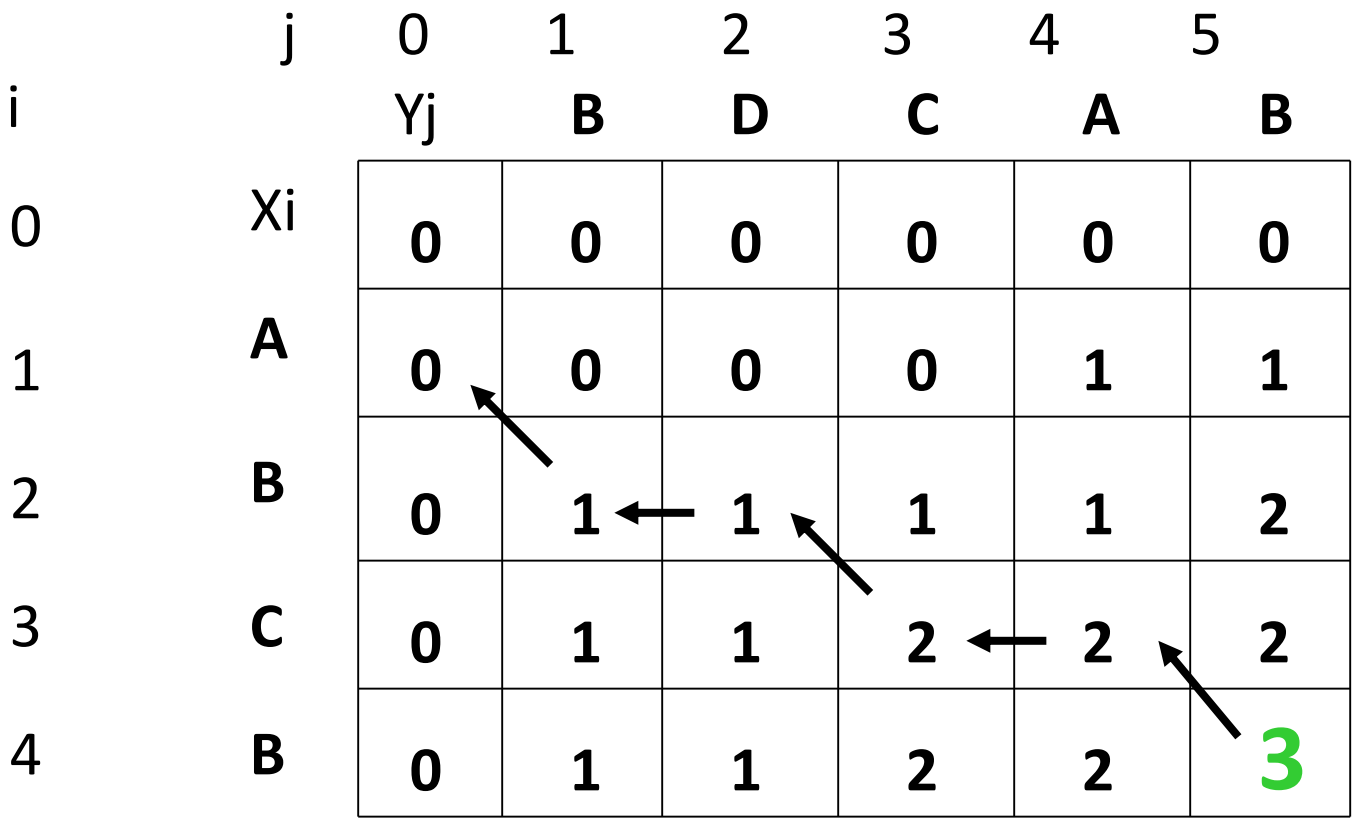
How to find actual LCS - continued

- Remember that

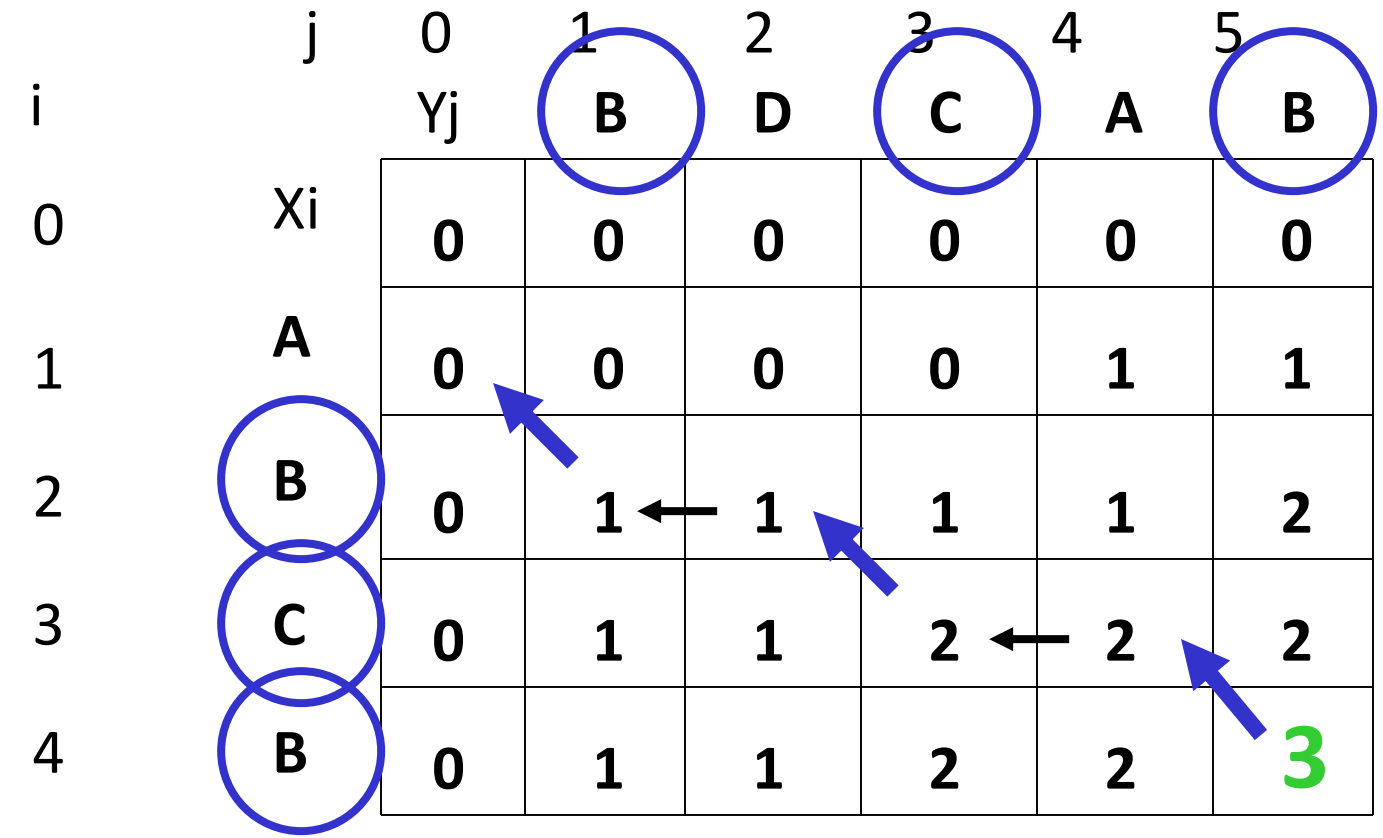
$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \text{otherwise} \end{cases}$$

- So we can start from c[m,n] and go backwards
- Whenever c[i,j] = c[i-1, j-1] + 1, since x[i]=y[j], remember x[i] (because x[i] is a part of LCS)
- When i=0 or j=0 (i.e. we reached the beginning), output remembered letters in reverse order.

Finding LCS


j 0 1 2 3 4 5
i
Yj B D C A B
0 Xi
1 A
2 B
3 C
4 B
0 0 0 0 0 0
0 0 0 0 1 1
0 1 1 1 1 2
0 1 1 2 2 2
0 1 1 2 2 3

Finding LCS (2)



LCS (reversed order): **B C B**

LCS (straight order): **B C B**

(this string turned out to be a palindrome)

Reconstruction time – O(m+n)

Example 4: All-pair Shortest Path

- Input: a directed graph G=(V,E) with $V = \{1, 2, ..., n\}$. The length of edge *e* is denoted by w(e), and it may be negative.

- Output: All-pair shortest path: for any two vertices v,u in V, what is the shortest path from v to u
  - we will only be interested in the *length* of that path.

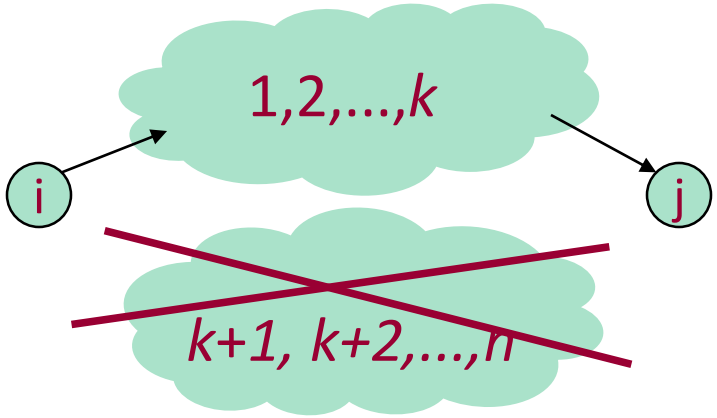
- Floyd Warshall Algorithm that solves this problem is actually a dynamic programming algorithm.

All-pair Shortest Path

Define

$$\delta^0(i,j) = \begin{cases} w(e) & \text{if } (i,j) \in E, \\ \infty & \text{if } (i,j) \notin E. \\ 0 & \text{if } i = j \end{cases}$$

Let $\delta^k(i,j)$ be the length of a shortest path from i to j among all paths which may pass through vertices 1,2,...,*k* but do not pass through vertices *k+1, k+2,...,n.*



Floyd -Warshall Algorithm
Dynamic Programming Solution:

1. Init $\delta^0(i, j)$ as defined earlier
2. $k \leftarrow 1$
3. For every $1 \leq i, j \leq n$ compute

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}.$$

4. If $k = n$, stop. If not, increment k and go to step 3.

Note: δ is a 3-dim table.
δ^0 includes the base cases, δ^n is the output.
δ^k is computed from δ^{k-1}.

Floyd -Warshall Algorithm

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}.$$

The shortest path from i to j which may pass through vertices 1,2,...,*k* but do not pass through vertices *k+1, k+2,...,n:*

1. Might not pass through vertex k, or
2. Might pass through k, and then it is composed by two already-computed shortest paths.

